

Exception Handling in Java

Exception handling in Java helps manage runtime errors using `try-catch` blocks. The main components are:

- **try:** Contains code that may throw an exception.
- **catch:** Catches and handles exceptions thrown in the `try` block.
- **finally:** Executes code regardless of whether an exception was thrown or not.
- **throw:** Used to explicitly throw an exception.
- **throws:** Declares exceptions a method might throw.

Basic Syntax:

```
try {  
  
} catch (ExceptionType e) {  
  
} finally {  
  
}
```

Examples:

1. Simple Try-Catch:

```
public class Example {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0;  
        } catch (ArithmeticException e) {  
            System.out.println("Error: Division by zero!");  
        }  
    }  
}
```

Output: Error: Division by zero!

2. Multiple Catch Blocks:

```
public class Example {  
    public static void main(String[] args) {  
        try {  
            String str = null;  
            System.out.println(str.length());  
        } catch (NullPointerException e) {  
            System.out.println("Null pointer error!");  
        } catch (ArithmeticException e) {  
            System.out.println("Arithmetic error!");  
        }  
    }  
}
```

Output: Null pointer error!

3. Finally Block:

```
public class Example {
    public static void main(String[] args) {
        try {
            int result = 10 / 2;
        } finally {
            System.out.println("Finally block always runs.");
        }
    }
}
```

Output: Finally block always runs.

4. Throwing an Exception:

```
public class Example {
    public static void main(String[] args) {
        try {
            checkAge(17);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }

    static void checkAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be 18 or older.");
        }
    }
}
```

Output: Age must be 18 or older.

5. Using Throws:

```
public class Example {
    public static void main(String[] args) {
        try {
            throwsException();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    static void throwsException() throws Exception {
        throw new Exception("Something went wrong!");
    }
}
```

Output: Something went wrong!

Conclusion

Java's exception handling ensures that errors are caught and managed gracefully, preventing crashes and allowing for proper cleanup and error messaging.